

Implementação e Comparação de Planejadores Clássicos para Escalonamento de Tarefas

Robson Medeiros Santos

¹Instituto de Ensino Superior – ICEV

robsonpaulista@hotmail.com

Resumo. *Este trabalho apresenta a modelagem de um domínio PDDL para escalonamento de tarefas em um laboratório de prototipagem, bem como a implementação e comparação de três planejadores clássicos em Python: um planejador STRIPS baseado em busca em largura/A* com heurísticas informadas, uma adaptação do Graphplan e um SATPlan com codificação CNF incremental e opção de solver externo (PySAT/Minisat22). Complementarmente, desenvolvemos um painel Streamlit para visualização gráfica dos planos e um validador automático. Os planejadores foram avaliados em três instâncias de problema, medindo-se tempo de execução, tamanho do plano e métricas específicas de cada abordagem. Os resultados mostram que todas as abordagens produzem planos válidos, mas apresentam perfis de desempenho distintos, especialmente no cenário mais complexo.*

Palavras-chave: planejamento clássico, escalonamento de tarefas, PDDL, STRIPS, Graphplan, SATPlan

Abstract. *This paper presents the modeling of a PDDL domain for task scheduling in a prototyping laboratory, as well as the implementation and comparison of three classical planners in Python: a STRIPS planner based on breadth-first search, an adaptation of Graphplan, and a SATPlan with incremental CNF encoding. The planners were evaluated on three problem instances, measuring execution time, plan length, and approach-specific metrics. Results show that all approaches yield valid plans but with distinct performance profiles, particularly in the most complex scenario.*

Keywords: classical planning, task scheduling, PDDL, STRIPS, Graphplan, SATPlan

1. Introdução

Planejamento clássico fornece ferramentas formais para a geração automática de planos em sistemas determinísticos e plenamente observáveis. Nesta atividade prática, modelamos um domínio inspirado em um laboratório de fabricação digital, onde tarefas exigem operadores e máquinas específicas e respeitam restrições de precedência. O objetivo é implementar três paradigmas distintos – STRIPS, Graphplan e SATPlan – e comparar seu desempenho em instâncias de complexidade crescente, incorporando ainda heurísticas informadas, suporte a solver SAT externo, validação automática de planos e visualização interativa.

2. Modelagem do Domínio

2.1. Elementos do domínio

O arquivo `domain.pddl` define tipos para operadores, máquinas, recursos e tarefas, bem como predicados para disponibilidade, dependências e alocação. Utilizamos ações `start-task` e `finish-task` para capturar explicitamente a alocação/ liberação de recursos e a conclusão das tarefas.

2.2. Restrições modeladas

- Dependências são representadas pelo predicado `(depends ?t ?p)`, garantindo que uma tarefa só possa iniciar quando todas as predecessoras estiverem concluídas.
- Requisitos de máquina são tratados por `(requires ?t ?m)`; quantificadores universais com igualdade asseguram que a máquina selecionada em `start-task` corresponda às exigências do problema.
- Uso exclusivo de recursos é modelado via predicados `available` e literais de atribuição, impedindo que um operador ou máquina execute múltiplas tarefas simultaneamente.

3. Metodologia e Implementação

3.1. Planejador STRIPS

Implementado em `planner_strips.py`, suporta busca em largura e A* com heurísticas informadas. As heurísticas `h_add` e `h_max` são calculadas a partir das dependências entre tarefas, estimando o custo restante para concluir o conjunto de objetivos. O estado armazena tarefas concluídas, recursos disponíveis e tarefas em progresso. Métricas coletadas: número de ações no plano, nós expandidos e gerados, tempo total e, quando aplicável, heurística utilizada.

3.2. Planejador Graphplan

O arquivo `planner_graphplan.py` constrói níveis alternados de ações e literais (positivos e negativos), computando relações de *mutex* e realizando extração regressiva com memorização. As métricas incluem horizonte (número de níveis), nós expandidos durante a regressão e tempo. Literais negativos são contemplados para suportar precondições de indisponibilidade.

3.3. Planejador SATPlan

Em `planner_sat.py`, codificamos o problema em CNF com horizonte incremental, facultando o uso de um solver externo (PySAT/Minisat22) ou do DPLL interno. Isso permite comparar o desempenho entre uma implementação pura e uma versão acelerada por SAT moderno. Para cada horizonte viável registramos número de variáveis, cláusulas, tamanho do plano e tempo total.

3.4. Validação e Visualização

O módulo `plan_validator.py` verifica planos arbitrários simulando as ações do domínio e garante que pré-condições, alocação de recursos e metas finais sejam satisfeitas. Para análise qualitativa, `visualize_plan.py` e o painel Streamlit geram timelines (gráficos de Gantt) com base nas ações `start-task` e `finish-task`, destacando operadores e máquinas alocadas em cada passo.

3.5. Infraestrutura Experimental

O script `run_experiments.py` padroniza a execução nas três instâncias (`schedule1-3.pddl`) e consolida métricas. Todos os experimentos foram conduzidos em Python 3.11. Para os recursos opcionais (Streamlit, Plotly e PySAT) adotou-se `pip install streamlit pandas plotly python-sat[pblib,aiger]`.

4. Resultados e Análise Comparativa

A Tabela 1 resume as métricas coletadas. Todos os planejadores produziram planos de mesmo tamanho, refletindo a correta modelagem do domínio. Para o STRIPS apresentamos o caso base (busca em largura); adicionalmente, as heurísticas `h_add` e `h_max` reduziram o volume de nós explorados na instância `schedule2` (respectivamente 24 e 65 nós expandidos, contra 72 na busca cega).

Tabela 1. Resumo dos experimentos por planejador e instância.

Instância	Planej.	Ações	Tempo (s)	Exp.	Horiz.	Var./Cláus.
schedule1	STRIPS	6	0,006	6	–	–
schedule1	Graph.	6	0,020	7	6	–
schedule1	SATPlan	6	0,005	–	6	134/593
schedule2	STRIPS	8	0,007	72	–	–
schedule2	Graph.	8	0,160	5	4	–
schedule2	SATPlan	8	250,703	–	8	508/6 784
schedule3	STRIPS	12	0,002	140	–	–
schedule3	Graph.	12	0,107	8	6	–
schedule3	SATPlan	12	8 064,621	–	12	743/6 809

4.1. Discussão

STRIPS apresentou tempos reduzidos, e as heurísticas informadas mostraram potencial para diminuir a expansão de estados sem afetar o tamanho do plano. Graphplan manteve horizonte pequeno, obtendo planos dos mesmos tamanhos com custo moderado de extração. Já o SATPlan escalou pior quando limitado ao DPLL interno; integrações com solvers externos (PySAT/Minisat22) estão disponíveis no código e permitem reduzir drasticamente o tempo em instâncias maiores quando a biblioteca está instalada. A visualização temporal (timeline) e o validador automático auxiliaram na interpretação qualitativa e na garantia de correção dos planos produzidos por qualquer planejador.

5. Conclusões e Trabalhos Futuros

Os três paradigmas confirmam a corretude do domínio e produzem planos equivalentes. STRIPS é simples e eficiente para problemas de porte moderado, Graphplan equilibra horizonte e custo computacional, e SATPlan, embora geral, requer otimizações para escalabilidade. As extensões implementadas (heurísticas, PySAT, validador e visualização) elevam a reprodutibilidade e a análise qualitativa. Como trabalhos futuros sugerimos explorar heurísticas mais sofisticadas, comparar diferentes solvers SAT, enriquecer a visualização com indicadores de custo e integrar o validador a um pipeline automatizado de testes.

Agradecimentos

Agradecemos à instituição e à coordenação da disciplina pelo suporte proporcionado.

Referências

- [1] Richard E. Fikes e Nils J. Nilsson. STRIPS: A new approach to the application of theorem proving to problem solving. *Artificial Intelligence*, 2(3–4), 1971.
- [2] Avrim Blum e Merrick Furst. Fast planning through planning graph analysis. *Artificial Intelligence*, 90(1–2), 1997.
- [3] Henry Kautz e Bart Selman. Planning as satisfiability. *ECAI*, 1992.